PhishReport Project Initial Details

INF1103 Phase 1 | Team P9-G04 | Current scope: 13 October 2026

Instructor approval and prior submission are unverified. This revised statement is not proof of sign-off.

Repository: https://github.com/lgf2111/INF1103-Project1-P9-G04

Our one-paragraph problem statement

We are building PhishReport to help non-specialist students and staff interpret suspicious email, SMS and chat messages and decide what to do next, especially after clicking a link, downloading a file or disclosing a password or one-time code. Users provide content manually or through an exported .eml file, optional sender/link/file context and their reported actions; we collect disclosure categories rather than actual secrets. Every new completed assessment requires an AI API response with validated phishing, credential-request and uncertainty findings and extracted contacts. Procedural rules combine these findings with reported exposure to produce an explained numeric response-priority score, LOW/MEDIUM/HIGH category and response checklist. We save the canonical message, channel and completed outcome to local JSON for retrieval. This supports practical phishing triage and harm reduction; it does not authenticate senders, execute attachments or guarantee safety.

Our problem and target users

Suspicious messages mix misleading identity, urgent requests and links across communication channels. Non-specialists may recognise something is wrong without knowing whether their prior actions require urgent response. We address that decision with understandable reasons and concrete advice rather than an unexplained classification. Our target users are students and staff who can supply content and report their actions accurately.

Our user inputs

Manual input accepts Email, SMS or Chat, optional sender, nonblank message, link/file presence and optional details. Exported .eml input supports a folder picker and browser uploader using the same parser. We include subject and inline plain-text/HTML as inert text; we do not access a mailbox or fetch embedded resources. We separately ask about clicks, downloads and password/OTP disclosure. We reject invalid choices, blank required fields, malformed/unsupported emails and resource excess, then re-prompt or cancel. Images, PDFs and attachments are outside scope.

Our use of AI

Our AI manager constructs a specific JSON-output prompt containing message and optional untrusted metadata. The API returns credential_request, suspicious and insufficient_context booleans, plus email, phone and IP lists. We validate envelope, JSON, exact schema/types and source occurrences before use. AI findings drive our business rules and are essential to assessment. Connection failures have bounded retries then explicit failure. Invalid or absent findings do not become successful fallback assessments or saved reports.

Our business rules

The first matching rule assigns 90/HIGH for disclosed passwords/OTPs, 80/HIGH for AI suspicion plus click/download, 60/MEDIUM for both AI suspicion and credential request, 45/MEDIUM for suspicion alone, 40/MEDIUM for credential request alone, 35/MEDIUM for interaction alone, 20/LOW for insufficient context alone, otherwise 10/LOW. The third rule combines two AI fields. All rules require valid AI first. HIGH starts at 70 and MEDIUM at 35. Scores express response policy, not probabilities.

Reasons identify findings and exposure. Advice covers independent verification, avoiding further interaction and password/service follow-up after disclosure. LOW does not guarantee safety. Logic is deterministic for identical input/findings; fresh-model repeatability and the API requirement for stored-history reads need instructor clarification.

Our architecture, storage and delivery

We implement io_manager.py, ai_manager.py, logic_manager.py and data_manager.py as procedural modules. All terminal input/output lives in I/O; AI validates findings; logic evaluates/scores/routes; data saves/loads/queries JSON. New reports store schema version 2, channel, canonical message and result. Atomic writes and corrupt-file recovery preserve bytes rather than silently replacing bad history. A Docker directory mount retains reports/settings/logs after recreation.

Our repository contains code, required offline hardcoded-AI logic tests, Docker configuration and engineering report. I use focused commits, issues, PRs and CI. Submission is via main. I must confirm instructor sign-off, Initial Details xSite submission, member/lab-in-charge access and clones, and each member's final Docker run. This scope document does not substitute for receipts or approval.